

AstroLearn Microservice Structure

AstroLearn is an educational application designed to help users learn about planets and space missions through content and quizzes. Its microservice architecture was developed to ensure modularity, scalability, and independent deployment of each component. The system follows key design patterns such as the **API Gateway**, **Service Discovery**, and **RESTful APIs**, which enable efficient communication between services.

This report presents the architecture, explaining how synchronous and asynchronous communications are used, how external data is integrated, and how each service manages its own data.

Gateway and synchronous communications

The API Gateway acts as a single entry point for the application, routing incoming HTTPS requests from web and mobile clients to the appropriate microservices (GeeksforGeeks, 2025). These communications are implemented through synchronous RESTful APIs (HTTPS), meaning that each request expects an immediate response (Bindu, 2023). Ozkaya (2021) and Mozilla (n.d.) states that each RESTful request uses specific **HTTP methods** that define the type of operation: **GET** to retrieve data, **POST** to submit data, **PUT** to update existing data, and **DELETE** to remove it. This approach suits AstroLearn because it is a lightweight educational application where users perform short actions such as reading content, submitting quizzes, or checking progress.

To prevent unauthorised access, the system includes both public and protected routes. The public route connects to the **Auth Service**, which allows users to sign in and obtain a JWT (JSON Web Token). During login, the Gateway sends the user's credentials through a **POST** request to the Auth Service. This allows the client to submit the required data for authentication. Once verified, the Auth Service responds with a JWT token, which the Gateway forwards to the client. An optional **GET** request can be used to confirm the current identity derived from the token, for example the user ID.

From this point on, when the user makes a new request, the client includes the token in the request header. The Gateway validates this token before forwarding any request to protected routes, ensuring that only authenticated users can access restricted services.

The protected services include:

The **User Profile service** which manages user information. For example, **GET** retrieves profile data, **PUT** updates user preferences, and **DELETE** can remove the user account.

The **Quiz service**, which uses **GET**, to display questions and allow users to review past sessions. Then, **POST** is used to submit the answers of the quiz and receive an immediate result.

Four **read-only services** can be accessed using **GET**: **Space Catalog**, contains general information sheets and facts about planets, stars, and missions; **Progress**, shows quiz results and levels achieved; **News**, is a library of space articles; and **Sky Events**, a calendar of observable celestial events.

Then, the **Notifications Service**, where users can read notifications (**GET**) and delete them (**DELETE**) only within the in-app notifications section, passing through the API Gateway. This design keeps notifications non-intrusive, allowing users to check or clear them when they choose, instead of receiving external alerts.

Moreover, the **Space Data Importer** is a backend service responsible for integrating external information from space agencies such as NASA or ESA. It periodically gets this data through **GET** requests, filters and organises it, and then sends the new content to **News** and **Sky Events** by **POST** requests.

This component is not connected to the API Gateway because it operates in the background and does not interact with users. This design keeps external API management isolated and prevents unnecessary traffic between the client and backend systems.

Asynchronous communications

Between some of these services, AstroLearn includes asynchronous communications. While synchronous APIs are used for direct user interactions, asynchronous interactions are adopted for background updates that do not require an immediate response. This allows services to continue working independently, improving scalability and reducing processing delays, while ensuring that each service focuses only on its own logic without depending directly on others. (Grunfeld, 2025)

Quiz Service triggers asynchronous events when users complete a quiz. An event such as “QuizCompleted” is sent to the **Progress Service**, which updates the user’s scores and level, and to the **Notifications Service**, which creates an alert inside the in-app notification section for both cases: when a quiz is completed or when a new level is achieved (event from the Progress Service).

Similarly, when new content is published, the **News Service** releases an “ArticlePublished” event, and the **Sky Events Service** emits an “EventUpcoming” event. Both events are

received by the **Notifications Service**, which stores and displays these messages in the appropriate section.

AstroLearn does not use a message broker because the number of events is limited and the application is small. Events are exchanged through internal asynchronous messages, as shown above, which is sufficient for a small-scale educational system. Overall, the asynchronous approach complements the synchronous API layer by decoupling background processes from user requests.

Service Discovery

In microservice systems, services can move, restart, or scale, so they can change position. Therefore, according to Rahman (2024) **Service Discovery** is essential, as it acts as a **registry**, that keeps track of all active microservices and their locations. Enabling services to locate and communicate with each other.

Each service **registers itself** in the registry and sends **heartbeat signals** to confirm it is active. When the **API Gateway** or another service needs to connect to a specific microservice, it performs a **lookup** request to find the most recent address. In AstroLearn, **Quiz**, **Progress**, **News**, and **Sky Events** perform lookups to locate the **Notifications Service** before sending asynchronous REST calls, while **Quiz** also looks up **Progress**, and the **Space Data Importer** looks up **News** and **Sky Events**.

This ensures that every service can be updated or redeployed while maintaining smooth communication across the system.

Database

Each of the following services: **Auth**, **User Profile**, **Space Catalog**, **News**, **Sky Events**, **Quiz**, **Progress**, and **Notifications** is designed to be **data-independent** within a shared database. This means that all services store their data in the same database; however, each one has its **own separate space**, where it stores and manages the information related to its specific function, so that no service can access another's data.

For example, the **Progress Service** handles quiz results and user achievements, without accessing data from other services. Keeping the data separated, it makes each service easier to scale, maintain, or update independently. This approach simplifies development for a small-scale prototype while preserving the modularity required by a microservices architecture (Brown, 2020).

In contrast, components such as the **API Gateway**, **Service Discovery**, and the **Space Data Importer** do not need a database, as they only route traffic, manage registration, or temporarily process data from external APIs.

References:

Bindu, C. (2023). *CloudNative Application Series 2: Synchronous Communication Pattern*.

Medium. <https://medium.com/@bindubc/cloudnative-application-series-2-synchronous-communication-pattern-505a57f07c51>

Brown, K. G. (2020). *Choosing the right databases for microservices*. Medium.

<https://kylegenebrown.medium.com/database-considerations-for-microservices-f575d5448203>

GeeksforGeeks. (2025). *What is API Gateway?* [https://www.geeksforgeeks.org/system-](https://www.geeksforgeeks.org/system-design/what-is-api-gateway-system-design/)

[design/what-is-api-gateway-system-design/](https://www.geeksforgeeks.org/system-design/what-is-api-gateway-system-design/)

Grunfeld, A. (2025). *Synchronous vs. asynchronous communication: What to use when*.

Riverside. <https://riverside.com/blog/synchronous-vs-asynchronous>

Mozilla. (n.d.). *HTTP Request Methods*. [https://developer.mozilla.org/en-](https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods)

[US/docs/Web/HTTP/Reference/Methods](https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods)

Ozkaya, M. (2021). *Microservices communications*. Medium. [https://medium.com/design-](https://medium.com/design-microservices-architecture-with-patterns/microservices-communications-f319f8d76b71)

[microservices-architecture-with-patterns/microservices-communications-f319f8d76b71](https://medium.com/design-microservices-architecture-with-patterns/microservices-communications-f319f8d76b71)

Rahman, J. (2024). *Understanding service discovery in microservices architecture*. Medium.

<https://medium.com/%40jeslurrahman/understanding-service-discovery-in-microservices-architecture-2098b10e7439>